# فصل ۱: نوع داده، عملگرها، اولویت و تبدیل نوع (زبان C)

**چرا این فصل؟** درس «مبانی کامپیوتر و برنامه‌سازی» در سرفصل دکتری ۱۴۰۶ (مجموعهٔ ۲۳۵۴) آمده، ولی در کنکور ارشد کامپیوتر درسی با این نام نبوده و سؤال سابقه ندارد. تیپ رایج این درس **«خروجی این کد چیست؟»** است و تقریباً همهٔ تله‌ها از همین فصل می‌آیند: تقسیم صحیح، اولویت عملگرها، اتصال کوتاه، و تبدیل نوع. **فرض‌ها:** C استاندارد، `int` ۴ بایتی، `char` ۱ بایتی (حالت رایج در تست‌ها). **اعتبار:** خروجی همهٔ کدهای این فصل با کامپایلر `gcc` اجرا و چک شده است. **زمان مطالعه:** یک بلوک ۹۰ دقیقه‌ای.

---

## درس ۱: انواع داده

| نوع | اندازهٔ رایج | نکته |
|---|---|---|
| `char` | ۱ بایت | در واقع یک عدد صحیح کوچک است |
| `short` | ۲ بایت | |
| `int` | ۴ بایت | |
| `long` | ۴ یا ۸ بایت | به سیستم بستگی دارد (لینوکس ۶۴ بیتی: ۸) |
| `double` / `float` | ۸ / ۴ بایت | `0.1` در C به‌طور پیش‌فرض `double` است |

### سه نکتهٔ تستی

۱. **`char` عدد است:** `'A'` برابر ۶۵، `'a'` برابر ۹۷ و `'0'` برابر ۴۸ است. پس `'a' - 'A' = 32` و `'7' - '0' = 7`.

۲. **`unsigned` دور می‌زند** (به پیمانهٔ $2^n$): اگر `unsigned char` را ۲۵۵ بگذاری و `++` کنی، صفر می‌شود. سرریز `signed` اما **رفتار تعریف‌نشده** است و تست درست آن را نمی‌پرسد.

۳. **`sizeof` عبارت داخلش را اجرا نمی‌کند:** بعد از `sizeof(x++)` مقدار x عوض **نمی‌شود**. همچنین در C، `sizeof('A')` برابر ۴ است (چون `'A'` از نوع int است)؛ در C++ برابر ۱ است.

### تمرین درس ۱ (خروجی چیست؟)

```
// 1
printf("%d %d %d", 'A', 'a' - 'A', '7' - '0');
// 2
unsigned char c = 250;  c = c + 10;  printf("%d", c);
// 3
int x = 7;  size_t s = sizeof(x++);  printf("%zu %d", s, x);
// 4
unsigned int u = 0;  u = u - 1;  printf("%u", u);
```

🔑 **جواب‌ها:** پاسخ‌نامهٔ انتهای فصل، بخش ۱ (درس ۱: انواع داده)

---

## درس ۲: عملگرهای حسابی و افزایشی

### تقسیم و باقی‌مانده

- تقسیم دو عدد صحیح، **صحیح** است و **به سمت صفر بریده می‌شود**: `7/2 = 3` و `-7/2 = -3`.
- **علامت `%` همان علامت عدد اول است:** `-7 % 2 = -1` و `7 % -2 = 1`.
- همیشه برقرار است: `a == (a/b)*b + a%b`.

### `++` و `--`

| شکل | معنی |
|---|---|
| `i++` | **اول مقدار فعلی** در عبارت استفاده می‌شود، بعد i یکی زیاد می‌شود |
| `++i` | اول i زیاد می‌شود، بعد مقدار جدید استفاده می‌شود |

**مثال:** `int i = 5; int j = i++ + 2;` ⇒ `j = 7` و `i = 6`.

⚠️ **رفتار تعریف‌نشده:** `i = i++;` یا `printf("%d %d", i++, i++);`. تغییر یک متغیر دو بار بدون «نقطهٔ ترتیب» خروجی قطعی ندارد. اگر در تستی دیدی، گزینهٔ «وابسته به کامپایلر / تعریف‌نشده» را در نظر بگیر.

### انتساب ترکیبی

در `a *= 3 + 4`، **کل سمت راست اول حساب می‌شود**، یعنی `a = a * (3+4)`. پس اگر a برابر ۲ باشد، نتیجه ۱۴ است نه ۱۰.

### تمرین درس ۲

```
// 1
printf("%d %d %d %d", -7/2, -7%2, 7%-2, 7/-2);
// 2
int i = 5;  int j = i++ + 2;  int k = ++i * 2;  printf("%d %d %d", i, j, k);
// 3
int a = 17, b = 5;  printf("%d %d", a/b*b + a%b, a % b * 2);
```

🔑 **جواب‌ها:** پاسخ‌نامهٔ انتهای فصل، بخش ۲ (درس ۲: عملگرهای حسابی و افزایشی)

---

## درس ۳: اولویت، شرکت‌پذیری و اتصال کوتاه

### جدول اولویت (از بالا به پایین؛ فقط آنچه در تست می‌آید)

| اولویت | عملگرها | جهت |
|---|---|---|
| ۱ | `()` `[]` `->` `.` و `++` `--` پسوندی | چپ به راست |
| ۲ | یکانی: `!` `~` `++` `--` `-` `*` `&` `sizeof` `(type)` | **راست به چپ** |
| ۳ | `*` `/` `%` | چپ به راست |
| ۴ | `+` `-` | |
| ۵ | `<<` `>>` | |
| ۶ | `<` `<=` `>` `>=` | |
| ۷ | `==` `!=` | |
| ۸ | `&` | |
| ۹ | `^` | |
| ۱۰ | `\|` | |
| ۱۱ | `&&` | |
| ۱۲ | `\|\|` | |
| ۱۳ | `?:` | **راست به چپ** |
| ۱۴ | `=` `+=` `-=` … | **راست به چپ** |

| اولویت | عملگرها | جهت |
|---|---|---|
| ۱۵ | `,` | چپ به راست |

**جملهٔ حفظی:** «یکانی، ضرب، جمع، شیفت، مقایسه، تساوی، بیتی (& ^ |)، منطقی (&& ||)، شرط، انتساب، کاما.»

## پنج تلهٔ معروف

۱. **`a < b < c` بازه را چک نمی‌کند.** اول `a<b` حساب می‌شود که ۰ یا ۱ است، بعد آن عدد با c مقایسه می‌شود. مثال: `5 > 4 > 3` برابر `(1) > 3` یعنی **۰** است.

۲. **بیتی از تساوی ضعیف‌تر است:** `x & 3 == 3` یعنی `x & (3==3)` یعنی `x & 1`.

۳. **شیفت از جمع ضعیف‌تر است:** `1 << 2 + 1` یعنی `1 << 3` یعنی ۸.

۴. **اتصال کوتاه:** در `A && B` اگر A نادرست باشد B **اجرا نمی‌شود**؛ در `A || B` اگر A درست باشد B اجرا نمی‌شود. پس `++` داخل B ممکن است اصلاً اجرا نشود.

۵. **`=` به جای `==`:** `if (x = 0)` x را صفر می‌کند و شرط همیشه نادرست است. عملگر کاما هم مقدار **آخرین** عبارت را برمی‌گرداند: `(1, 2, 3)` برابر ۳ است.

## تمرین درس ۳

```
// 1
int a = 3, b = 4, c = 5;  printf("%d %d", a < b < c, c > b > a);
// 2
int x = 6 & 3 == 3;  printf("%d", x);
// 3
int i = 0, j = 0;
if (i++ && j++) ;   printf("%d %d ", i, j);
if (i++ || j++) ;   printf("%d %d", i, j);
// 4
printf("%d %d", !5 == 0, 1 << 2 + 1);
// 5
int y = 5;  y = y > 3 ? y << 1 : y >> 1;  printf("%d", y);
```

🔑 **جواب‌ها:** پاسخ‌نامهٔ انتهای فصل، بخش ۳ (درس ۳: اولویت، شرکت‌پذیری و اتصال کوتاه)

---

# درس ۴: تبدیل نوع

## قواعد

۱. **نوع عملیات از روی عملوندها تعیین می‌شود، نه از روی متغیری که نتیجه در آن ذخیره می‌شود.** در `double d = 5/2;` تقسیم صحیح است و `d = 2.0`.

۲. اگر یکی از عملوندها اعشاری باشد، عملیات اعشاری می‌شود: `(double)5/2 = 2.5` ، ولی `(double)(5/2) = 2.0`.

۳. **تبدیل اعشاری به صحیح به سمت صفر می‌بُرد:** `(int)2.7 = 2` و `(int)-2.7 = -2` .

۴. **مقایسهٔ signed با unsigned:** عدد signed به unsigned تبدیل می‌شود. پس `-1 < 1u` **نادرست** است، چون -۱ به `4294967295` تبدیل می‌شود.

۵. **`float` با `double`:** `float f = 0.1;` و بعد `f == 0.1` نادرست است، چون `0.1` در float و double با دقت متفاوت ذخیره می‌شود.

### تمرین درس ۴

```
// 1
double d = 5/2;  printf("%f %.1f %.1f", d, (double)5/2, (double)(5/2));
// 2
printf("%s", (-1 < 1u) ? "yes" : "no");
// 3
int x = 'a' + 1.5;  printf("%d", x);
// 4
printf("%.1f %.1f", 3/2*2.0, 3*2.0/2);
// 5
float f = 0.1;  printf("%s", f == 0.1 ? "equal" : "not equal");
```

🔑 **جواب‌ها:** پاسخ‌نامهٔ انتهای فصل، بخش ۴ (درس ۴: تبدیل نوع)

---

## 📝 آزمون فصل ۱

این درس در کنکور ارشد کامپیوتر سؤال سابقه ندارد. پس این ۱۰ تست **تألیفی** هستند و در قالب تست کنکور نوشته شده‌اند. خروجی همهٔ کدها با `gcc` اجرا و تأیید شده است.

**زمان: ۱۵ دقیقه.** خروجی هر کد چیست؟

**۱.**

```
int a = 10, b = 3;
printf("%d %d", a / b, -a % b);
```

۱) `3 1` ۲) `3 -1` ۳) `4 -1` ۴) `3 2`

**۲.**

```
int x = 5, y = 0;
if (x > 3 || ++y) x++;
printf("%d %d", x, y);
```

۱) `6 1` ۲) `6 0` ۳) `5 1` ۴) `5 0`

**۳.**

```
int a = 1, b = 2, c = 3;
printf("%d", a < b < c == c > b > a);
```

۱) `0` ۲) `1` ۳) `3` ۴) خطای کامپایل

**۴.**

```
unsigned int u = 1;
int i = -1;
printf("%s", i < u ? "less" : "not less");
```

۱) `less` ۲) `not less` ۳) خطای کامپایل ۴) رفتار تعریف‌نشده

**۵.**

```
double d = 7 / 2 + 7 / 2.0;
printf("%.1f", d);
```

۱) `7.0` ۲) `6.5` ۳) `6.0` ۴) `7.5`

**۶.**

```
int x = 0x0F;
printf("%d", x & 0x3C | 1 << 4);
```

۱) `28` ۲) `12` ۳) `208` ۴) `13`

**۷.** (فرض: `int` ۴ بایتی)

```
int i = 3;
int j = sizeof(i++) + i;
printf("%d", j);
```

۱) `7` ۲) `8` ۳) `4` ۴) خطای کامپایل

**۸.**

```
char c = 'B';
c = c + ('a' - 'A');
printf("%c %d", c, c - 'a');
```

۱) `b 1` ۲) `B 1` ۳) `b 33` ۴) `b 0`

**۹.**

```
int a = 5, b;
b = (a++, a * 2);
printf("%d %d", a, b);
```

۱) `6 12` ۲) `5 10` ۳) `6 10` ۴) `5 12`

**۱۰.**

```
int k = 0;
int r = (k = 3) && (k = 0) || (k = 5);
printf("%d %d", r, k);
```

۱) `1 5` ۲) `0 0` ۳) `1 3` ۴) `0 5`

> 🔑 **جواب‌ها و راه‌حل:** پاسخ‌نامهٔ انتهای فصل، بخش ۵ (آزمون فصل ۱)

---

## 🗂 کارت خلاصهٔ فصل ۱

- `char` عدد است: A=65، a=97، 0=48. `unsigned` دور می‌زند. `sizeof` اجرا نمی‌کند.
- `/` صحیح به سمت صفر می‌بُرد. علامت `%` همان علامت عدد اول است.
- `i++` اول مقدار قدیمی را می‌دهد. دو بار تغییر یک متغیر در یک عبارت رفتار تعریف‌نشده است.
- اولویت: یکانی > `* / %` > `+ -` > شیفت > مقایسه > تساوی > `&` > `^` > `|` > `&&` > `||` > `?:` > `=` > `,`
- `a<b<c` بازه نیست. `&&` و `||` اتصال کوتاه دارند.
- نوع عملیات را عملوندها تعیین می‌کنند. signed در مقایسه با unsigned تبدیل می‌شود. float را با `==` مقایسه نکن.

# 🔑 پاسخ‌نامهٔ فصل

## بخش ۱: درس ۱: انواع داده

۱. `65 32 7`

۲. `4` (۲۶۰ به پیمانهٔ ۲۵۶)

۳. `4 7` (x اجرا نشد)

۴. `4294967295` (یعنی $2^{32} - 1$)

## بخش ۲: درس ۲: عملگرهای حسابی و افزایشی

۱. `-3 -1 1 -3`

۲. `7 7 14` (بعد از خط دوم i=6 و j=7؛ بعد `i++` آن را ۷ می‌کند و `k = 14`)

۳. `4 17 4` (`*` و `%` هم‌اولویت‌اند و از چپ به راست حساب می‌شوند: `4 = 2*(17%5)`)

## بخش ۳: درس ۳: اولویت، شرکت‌پذیری و اتصال کوتاه

۱. `1 0`

۲. `0` (`6 & 1 = 0`)

۳. `0 1 0 2 0`. در شرط اول، `i++` مقدار ۰ را برمی‌گرداند، پس `j++` اجرا نمی‌شود. در شرط دوم، `i++` مقدار ۱ را برمی‌گرداند، پس باز `j++` اجرا نمی‌شود.

۴. `1 8` (`!5 = 0` و `0 == 0` یعنی ۱)

۵. `10`

## بخش ۴: درس ۴: تبدیل نوع

۱. `2.000000 2.5 2.0`

۲. `no`

۳. `98` (`97 + 1.5 = 98.5` که بریده می‌شود)

۴. `2.0 3.0`. در اولی `3/2` اول حساب می‌شود و ۱ است. در دومی `3*2.0 = 6.0` و بعد تقسیم بر ۲.

۵. `not equal`

## بخش ۵: آزمون فصل ۱

| سؤال | ۱ | ۲ | ۳ | ۴ | ۵ | ۶ | ۷ | ۸ | ۹ | ۱۰ |
|---|---|---|---|---|---|---|---|---|---|---|
| جواب | ۲ | ۲ | ۱ | ۲ | ۲ | ۱ | ۱ | ۱ | ۱ | ۱ |

۱. `10/3 = 3` . منفی یکانی اولویت بالاتری دارد، پس `-a % b` یعنی `(-10) % 3` که برابر `-1` است (علامت عدد اول).

۲. `x > 3` درست است، پس `++y` به خاطر اتصال کوتاه اجرا نمی‌شود. خروجی `6 0` .

۳. مقایسه‌ها از `==` قوی‌ترند. سمت چپ: `(1<2)=1` و `1<3` می‌شود ۱. سمت راست: `(3>2)=1` و `1>1` می‌شود ۰. پس `1 == 0` یعنی **۰**.

۴. -۱ به unsigned تبدیل می‌شود و `4294967295` می‌شود، که از ۱ بزرگ‌تر است. خروجی `not less` . این رفتار کاملاً تعریف‌شده است.

۵. `7/2 = 3` (صحیح) و `7/2.0 = 3.5` . جمع **6.5**.

۶. ترتیب اولویت: شیفت، بعد `&` ، بعد `|` . `1<<4 = 16` و `0x0F & 0x3C = 0x0C = 12` . پس `12 | 16 = 28` .

۷. `sizeof` اجرا نمی‌کند، پس i همان ۳ می‌ماند. جواب `4 + 3 = 7` .

۸. `'B' + 32 = 'b'` و `'b' - 'a' = 1` .

۹. کاما نقطهٔ ترتیب است: اول `a++` اجرا می‌شود و a می‌شود ۶، بعد `b = 6*2 = 12` .

۱۰. `&&` از `||` قوی‌تر است. `(k=3)` مقدار ۳ (درست) دارد، `(k=0)` مقدار ۰ (نادرست)، پس `&&` می‌شود ۰. بعد `(k=5)` اجرا می‌شود که درست است، پس `r = 1` و `k = 5` . چون `&&` و `||` نقطهٔ ترتیب دارند، رفتار تعریف‌شده است.